

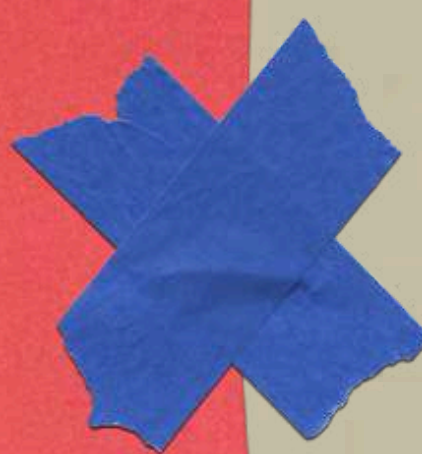
PROJETO

6

UFRPONG

POR PEDRO AILTON
BSI 2º PERÍODO
PROFESSOR LUCAS CAMBUIM





TÓPICOS A ABORDAR

Métodos criados



Movimentação

**Mais detalhes da
implementação**

Execução do Programa



MÉTODOS CRIADOS



PONG ORIGINAL

Método initialScreen()

Parte 1

```
method void initialScreen() {  
    var char key;  
  
    let key = 0;  
  
    do Screen.clearScreen();  
    do Output.moveCursor(11, 18);  
    do Output.printString("UFRPONG");  
    do Output.moveCursor(13, 15);  
    do Output.printString("Pressione qualquer tecla para jogar");  
  
    while (key = 0) {  
        let key = Keyboard.keyPressed();  
    }  
  
    // Espera soltar a tecla para não pular a próxima tela  
    while (~(key = 0)) {  
        let key = Keyboard.keyPressed();  
    }  
}
```


PONG ORIGINAL

Método initialScreen()

Parte 2

```
do Screen.clearScreen();
do Output.moveCursor(11, 18);
do Output.printString("Escolha o tema do jogo:");
do Output.moveCursor(13, 15);
do Output.printString("c -> claro | e -> escuro");

let key = 0;

while (key = 0) {
  let key = Keyboard.keyPressed();
}

// se a tecla presionada for "c"
if (key = 99){
  let isDarkMode = false; // Modo Claro
}
else {
  // se a tecla pressionada for "e"
  if (key = 101) {
    let isDarkMode = true; // Modo Escuro
  }
}

// Aplica o tema aos objetos
// Se DarkMode (Fundo Preto), objetos são Brancos (false)
// Se LightMode (Fundo Branco), objetos são Pretos (true)
do bat_cima.setColor(~isDarkMode);
do bat_baixo.setColor(~isDarkMode);
do ball.setColor(~isDarkMode);

// Prepara o fundo
do drawBackground();
```



PONG ORIGINAL

Método finalScreen()

```
method void finalScreen() {

    let key = 0;

    do Output.moveCursor(9, 18); // issue: centralizar essas duas mensagens na tela
    do Output.printString("Game Over. P Perdeu");
    do Output.moveCursor(9, 30); // Espaço de caractere logo depois do P, para completar "P2" ou "P1"
    do Output.printInt(perdedor);
    do Output.moveCursor(11, 18); // issue: centralizar essas duas mensagens na tela
    do Output.printString("Pressione ESC para Sair");
    do Output.moveCursor(13, 15);
    do Output.printString("Pressione SPACE para Jogar Novamente");

    while(~(key = KEY_ESC) & ~(key = KEY_SPACE)) {
        let key = Keyboard.keyPressed();
    }
    // Se ESC for pressionado, o repetir também acaba
    if (key = KEY_ESC) {
        let repetir = false; // variável de controle de repetição define que o run termine definitivamente
    }
    else {
        if (key = KEY_SPACE) {
            let exit = false; // variável de controle reinicia o exit

            do resetRound(); // reseta as variáveis e redesenha os elementos da tela

            // Imprime o placar
            do Output.moveCursor(22,0);
            do Output.printString("Score: P1: ");
            do Output.printInt(p1Score);
            do Output.printString(" / P2: ");
            do Output.printInt(p2Score);
        }
    }
    return;
}
```

PONG ORIGINAL

Método gameStep()

Vou abordar mais
quando eu for tratar
da movimentação dos
elementos do jogo

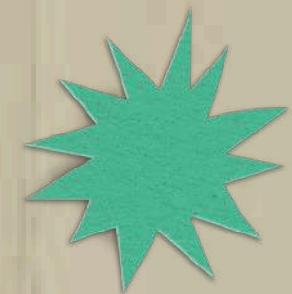
```
method void gameStep() {  
    // duas movimentações da raquete de cima  
    do bat_cima.move();  
    do bat_cima.move();  
  
    // duas movimentações da raquete de baixo  
    do bat_baixo.move();  
    do bat_baixo.move();  
  
    do moveBall(); // a bola se move, a partir dos métodos Pong  
    do moveBall(); // a bola se movimenta duas vezes por passo p  
  
    // um tempo de espera maior dá mais fluidez ao movimento, um  
    do Sys.wait(10); // muda a velocidade do jogo (em milissegun  
    return;  
}
```



PONG ORIGINAL

Método run()

```
while (~exit & ~repetir) {  
    // 1. LEITURA DE INPUT  
    let key = Keyboard.keyPressed();  
  
    // Lógica de controle  
    // Jogador 1 (Bat Cima) - Teclas A(97) e D(100)  
    if (key = 97) { do bat_cima.setDirection(1); } // Esquerda  
    else {  
        if (key = 100) { do bat_cima.setDirection(2); } // Direita  
        else { do bat_cima.setDirection(0); } // Parado (se não for nada)  
    }  
  
    // Jogador 2 (Bat Baixo) - Setas Esq(130) e Dir(132)  
    if (key = 130) { do bat_baixo.setDirection(1); } // Esquerda  
    else {  
        if (key = 132) { do bat_baixo.setDirection(2); } // Direita  
        else { do bat_baixo.setDirection(0); } // Parado  
    }  
  
    // Tecla ESC para sair  
    if (key = KEY_ESC) { let exit = true; }  
  
    // 1 Passo  
    do gameStep();  
  
    // se a bola já tiver colidido com a parede superior ou inferior  
    if (exit & ~(perdedor = 0)) {  
        if (perdedor = 2) { let p1Score = p1Score + 1; }  
        else { let p2Score = p2Score + 1; }  
  
        do ball.explodeAnimation();  
        do finalScreen();  
    }  
}
```



PONG ORIGINAL

Método resetRound()

```
method void gameStep() { ...
}

/**
Reseta as variáveis de controle do jogo e redesenha os elementos da tela do jogo
*/
method void resetRound() {
    if (~(ball = null)) {
        do ball.dispose(); // Garante que não haja vazamento de memória
    }

    let ball = Ball.new(253, 222, 0, 511, 16, 229);
    do ball.setColor(~isDarkMode);
    do ball.setDestination(200, -100);

    let wall = 0;
    let lastWall = 0;
    let perdedor = 0;

    do drawBackground();
    return;
}
```

PONG ORIGINAL

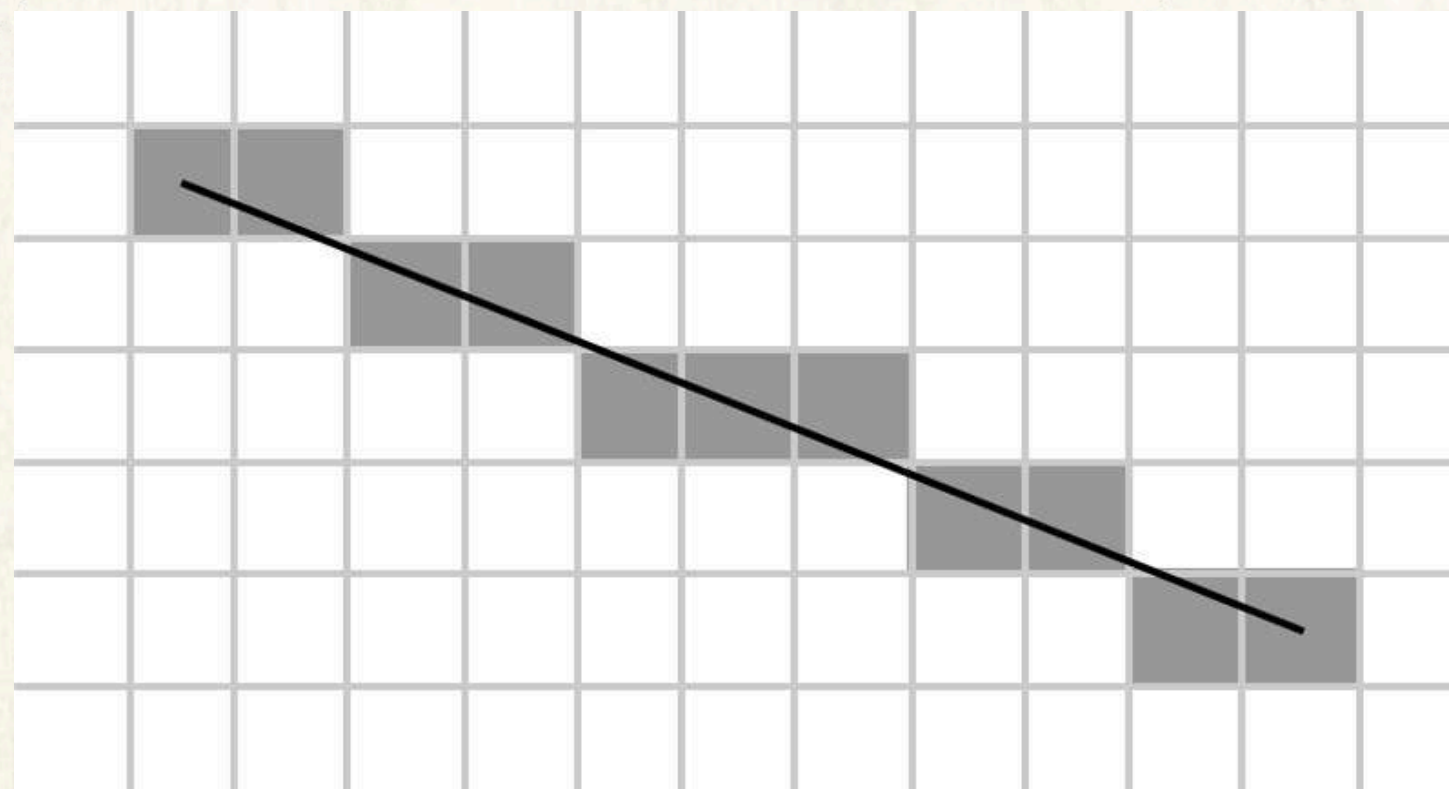
Método drawBackground()

```
method void drawBackground() {  
    if (isDarkMode) {  
        do Screen.setColor(true);  
        do Screen.drawRectangle(0, 0, 511, 255); // Pinta a tela toda de preto  
    }  
    else {  
        do Screen.clearScreen(); //Limpa toda a tela para ficar branca  
    }  
  
    // Desenha as barras (paredes)  
    // A cor das barras deve ser igual à dos objetos (contrária ao fundo)  
    do Screen.setColor(~isDarkMode);  
    do Screen.drawRectangle(0, 238, 511, 240); // Barra inferior  
    do Screen.drawRectangle(0, 4, 511, 6);      // Barra superior  
  
    // Redesenha os objetos para garantir que apareçam sobre o fundo  
    do bat_cima.show();  
    do bat_baixo.show();  
    do ball.show();  
  
    return;  
}
```


MOVIMENTAÇÃO

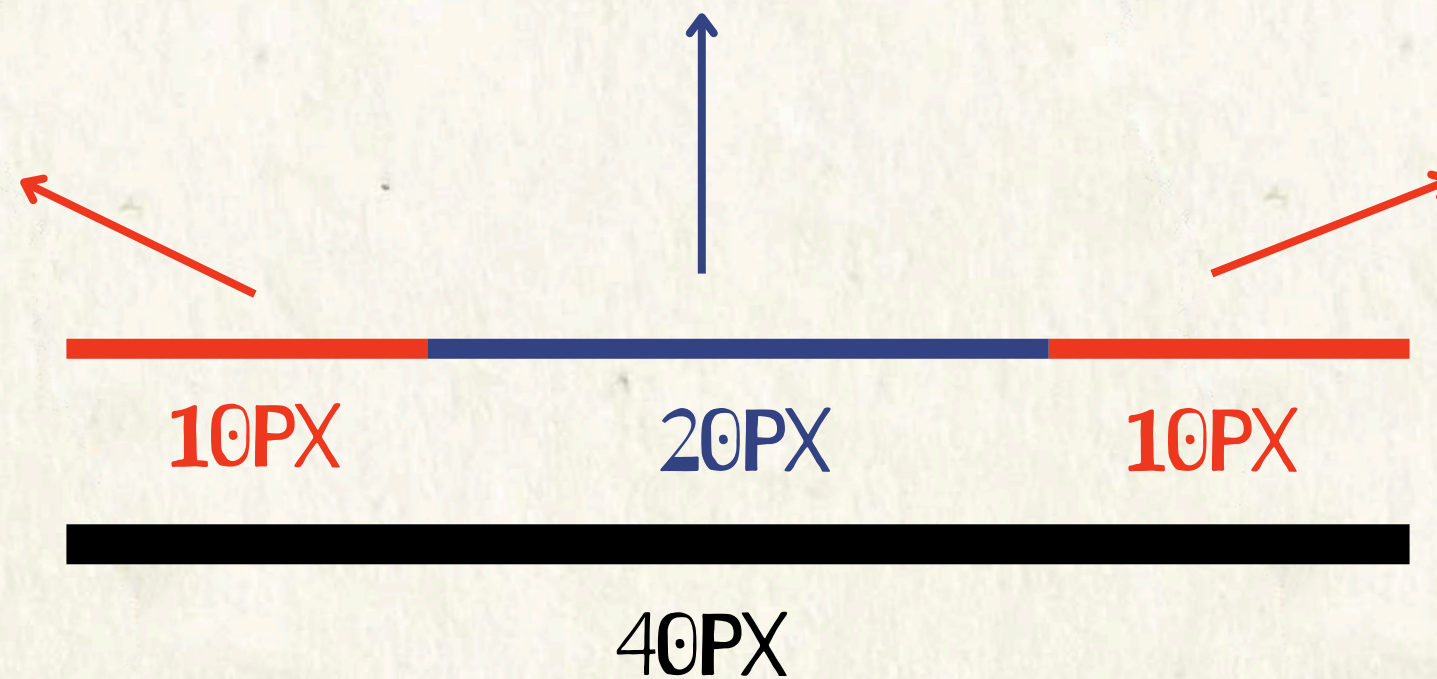


MOVE (ALGORITMO DE BRESENHAM)



IMPLLEMENTADO NO MÉTODO MOVE NA CLASSE BALL EM BALL.JACK

REBOTE

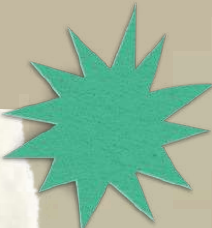


IMPLLEMENTADO NO MÉTODO BOUNCE NA CLASSE BALL EM BALL.JACK



REGULANDO OS

PARÂMETROS



Foi testado inicialmente apenas um movimento de cada elemento por step e um sys.wait menor

Dessa forma, ficou muito mais fluído

```
method void gameStep() {  
    // duas movimentações da raquete de cima  
    do bat_cima.move();  
    do bat_cima.move();  
  
    // duas movimentações da raquete de baixo  
    do bat_baixo.move();  
    do bat_baixo.move();  
  
    do moveBall(); // a bola se move, a parti  
    do moveBall(); // a bola se movimenta duas  
  
    // um tempo de espera maior dá mais fluide  
    do Sys.wait(10); // muda a velocidade do j  
    return;  
}
```

MAIS DETALHES




```
method void explodeAnimation() {
    var int memAddress;
    var int location;

    // y * 32: Como cada linha tem 32 palavras de
    // x / 16: Como cada endereço guarda 16 pixels
    let location = ((Math.multiply(y, 32)) + (Math

do hide(); // apaga a bola

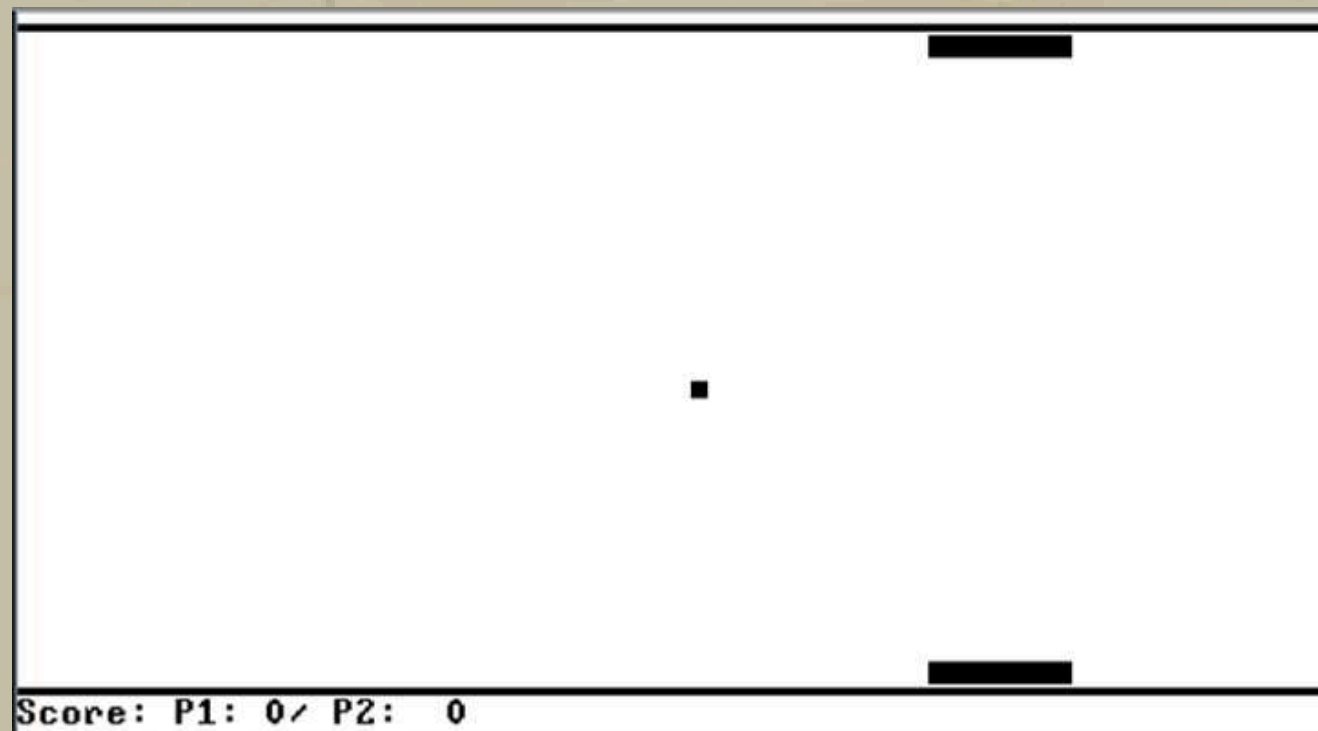
// FRAMES DESENVOLVIDOS NO BITMAP.EDITOR
// Frame 1
let memAddress = 16384+location;
do Memory.poke(memAddress+0, 0);
do Memory.poke(memAddress+32, 0);
do Memory.poke(memAddress+64, 0);
do Memory.poke(memAddress+96, 384);
do Memory.poke(memAddress+128, 576);
do Memory.poke(memAddress+160, 1440);
do Memory.poke(memAddress+192, 2640);
do Memory.poke(memAddress+224, 5160);
do Memory.poke(memAddress+256, 2640);
do Memory.poke(memAddress+288, 1440);
do Memory.poke(memAddress+320, 576);
do Memory.poke(memAddress+352, 384);
do Memory.poke(memAddress+384, 0);
do Memory.poke(memAddress+416, 0);
do Memory.poke(memAddress+448, 0);
do Memory.poke(memAddress+480, 0);
```

ANIMAÇÃO DE EXPLOÇÃO

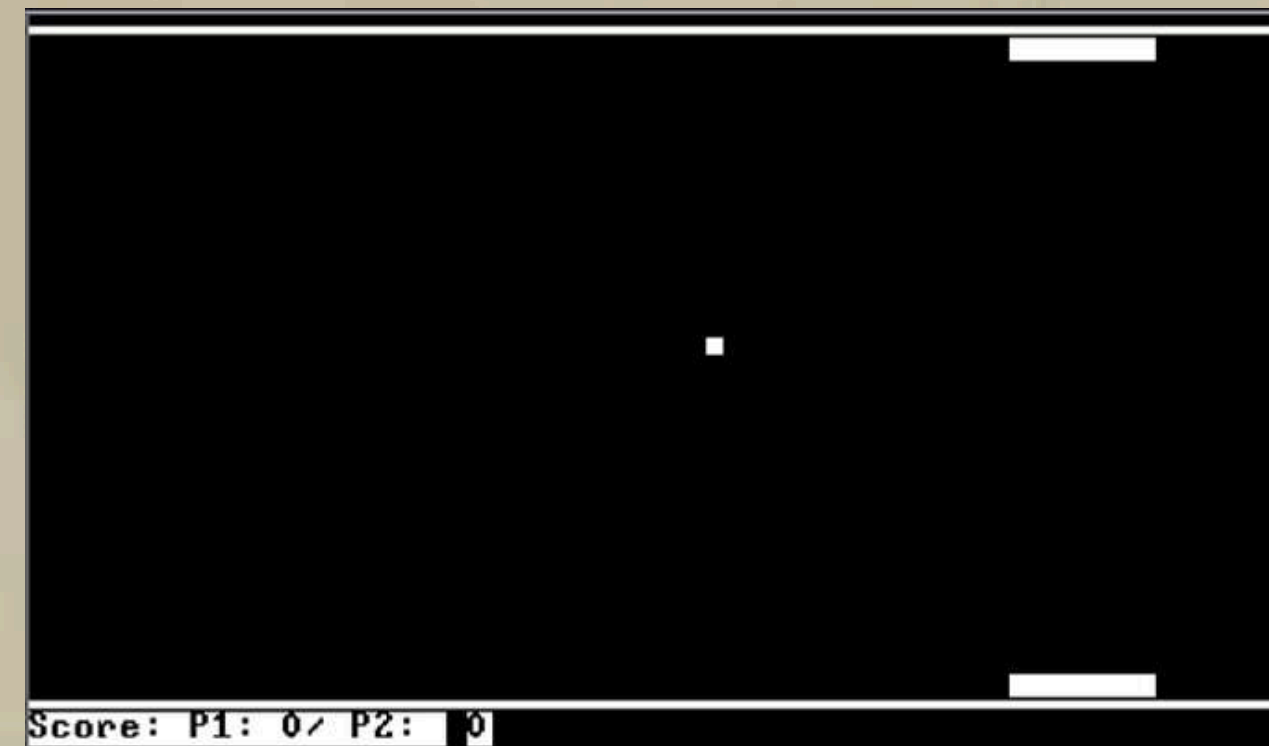
Sprite de 16x16 bits
feito com o uso do bitmap
editor disponibilizado no
arquivos disponíveis pelo
site nand to tetris, e
sugerido pelo professor.

ee

TEMAS



Claro



Escuro



TEMAS:

CLARO E ESCURO

```
// se a tecla presionada for "c"
if (key = 99){
    let isDarkMode = false; // Modo Claro
}
else {
    // se a tecla pressionada for "e"
    if (key = 101) {
        let isDarkMode = true; // Modo Escuro
    }
}
```

```
// Aplica o tema aos objetos
// Se DarkMode (Fundo Preto), objetos são Brancos (false)
// Se LightMode (Fundo Branco), objetos são Pretos (true)
do bat_cima.setColor(~isDarkMode);
do bat_baixo.setColor(~isDarkMode);
do ball.setColor(~isDarkMode);

// Prepara o fundo
do drawBackground();

return;
```

```
method void setColor(boolean new_color) {
    let color = new_color;
    return;
}
```

método **setColor()** dentro das classes **Bat** e **Ball**

Variável **isDarkMode** presente na classe **PongGame**

EXECUÇÃO





OBRIGADO!



LINK DO REPOSITÓRIO NO GITHUB